

Student Name: Tushar Panchal

En. No. : 21162101014

Branch: CBA

Batch: 71

Subject: ML (Machine Learning)

EXPERIMENT 8

For Multiclass classification on Drug prediction dataset, Binary classification on Credit card fraud detection dataset design :

1. Use following models: logistic regression, knn, decision tree and ANN
2. Calculate Precision, recall, accuracy etc.

Regression: Metro Interstate Traffic Volume Dataset :

1. User multi linear regression, polynomial regression and ANN
2. Calculate MSE, R2 score etc.

Drug Prediction Data

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

data = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/ML_7/p6_drug200.csv")
print("\nData Preview : \n")
print(data.head(10))
```

Data Preview :

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY
5	22	F	NORMAL	HIGH	8.607	drugX
6	49	F	NORMAL	HIGH	16.275	drugY
7	41	M	LOW	HIGH	11.037	drugC
8	60	M	NORMAL	HIGH	15.171	drugY
9	43	M	LOW	NORMAL	19.368	drugY

```
In [ ]: print("\nDataset Information:\n")
data.info()

label_encoder = LabelEncoder()
for col in ['Sex', 'BP', 'Cholesterol', 'Drug']:
    data[col] = label_encoder.fit_transform(data[col])

# Define features (X) and target (y)
X = data.drop('Drug', axis=1)
y = data['Drug']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

print("\nData preprocessing completed.")
print(f"Training set size: {X_train.shape}, Testing set size: {X_test.shape}")
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Age             200 non-null    int64
1   Sex             200 non-null    object
2   BP              200 non-null    object
3   Cholesterol     200 non-null    object
4   Na_to_K         200 non-null    float64
5   Drug            200 non-null    object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```

Data preprocessing completed.

Training set size: (140, 5), Testing set size: (60, 5)

```
In [ ]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling applied for improved model convergence.")
```

Feature scaling applied for improved model convergence.

```
In [ ]: from imblearn.over_sampling import SMOTE

# Use SMOTE to oversample the minority classes
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train_scaled, y_train)

print("Applied SMOTE to balance the dataset.")
```

Applied SMOTE to balance the dataset.

```
In [ ]: models = {
    ' Logistic Regression': LogisticRegression(solver='saga', max_iter=3000, r
    ' K-Nearest Neighbors': KNeighborsClassifier(),
    ' Decision Tree': DecisionTreeClassifier(),
    ' Artificial Neural Network': MLPClassifier(hidden_layer_sizes=(10,), max_
}

metrics = {}

# Train and evaluate models
for model_name, model in models.items():
    model.fit(X_train_balanced, y_train_balanced) # Use balanced dataset
    y_pred = model.predict(X_test_scaled)

    # Metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)

    # Store metrics in dict
    metrics[model_name] = {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'confusion_matrix': cm
    }

print("Training completed!")
```

Training completed!

```
In [ ]: from tabulate import tabulate

#Results
print("\nModel Performance Metrics:\n")
for model_name, metric in metrics.items():
    print(f"\n{model_name} : \n")
    print(f"Confusion Matrix : \n{metric['confusion_matrix']}\n")
    print("-" * 80)

comparison_table = [
    [model,
     f"{metrics[model]['accuracy']:.4f}",
     f"{metrics[model]['precision']:.4f}",
     f"{metrics[model]['recall']:.4f}",
     f"{metrics[model]['f1_score']:.4f}"]
    for model in metrics
```

```
]

print("\nComparison of Models : \n")
print(tabulate(
    comparison_table,
    headers=["Model", "Accuracy", "Precision", "Recall", "F1 Score"],
    tablefmt="fancy_grid"
))
```

Model Performance Metrics:

 Logistic Regression :

Confusion Matrix :

```
[[ 7  0  0  0  0]
 [ 0  3  0  0  0]
 [ 0  0  5  0  1]
 [ 0  0  0 18  0]
 [ 1  2  0  1 22]]
```

 K-Nearest Neighbors :

Confusion Matrix :

```
[[ 7  0  0  0  0]
 [ 0  3  0  0  0]
 [ 0  0  6  0  0]
 [ 0  1  0 17  0]
 [ 2  3  2  3 16]]
```

 Decision Tree :

Confusion Matrix :

```
[[ 7  0  0  0  0]
 [ 0  3  0  0  0]
 [ 0  0  6  0  0]
 [ 0  0  0 18  0]
 [ 0  0  0  0 26]]
```

 Artificial Neural Network :

Confusion Matrix :

```
[[ 3  0  3  0  1]
 [ 0  0  1  0  2]
 [ 0  0  5  0  1]
 [ 0  0 12  3  3]
 [ 0 14  1  5  6]]
```

Comparison of Models :

Model	Accuracy	Precision	Recall	F1 Score
 Logistic Regression	0.9167	0.9308	0.9167	0.9183
 K-Nearest Neighbors	0.8167	0.8755	0.8167	0.8164

 Decision Tree	1	1	1	1	
 Artificial Neural Network	0.2833	0.4519	0.2833	0.3083	

Credit Card Fraud Detection Data

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
In [ ]: df_fraud = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ML_7/p5_creditcard_fraud.csv')
print("\nData Preview : \n", df_fraud.head(10))
print("\nDataset Info : \n")
df_fraud.info()
```

Data Preview :

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	\
0	1	20000.0	2	2	1	24	2	2	-1	-1	
1	2	120000.0	2	2	2	26	-1	2	0	0	
2	3	90000.0	2	2	2	34	0	0	0	0	
3	4	50000.0	2	2	1	37	0	0	0	0	
4	5	50000.0	1	2	1	57	-1	0	-1	0	
5	6	50000.0	1	1	2	37	0	0	0	0	
6	7	500000.0	1	1	2	29	0	0	0	0	
7	8	100000.0	2	2	2	23	0	-1	-1	0	
8	9	140000.0	2	3	1	28	0	0	2	0	
9	10	20000.0	1	3	2	35	-2	-2	-2	-2	

	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	\
0	...	0.0	0.0	0.0	0.0	689.0	0.0	
1	...	3272.0	3455.0	3261.0	0.0	1000.0	1000.0	
2	...	14331.0	14948.0	15549.0	1518.0	1500.0	1000.0	
3	...	28314.0	28959.0	29547.0	2000.0	2019.0	1200.0	
4	...	20940.0	19146.0	19131.0	2000.0	36681.0	10000.0	
5	...	19394.0	19619.0	20024.0	2500.0	1815.0	657.0	
6	...	542653.0	483003.0	473944.0	55000.0	40000.0	38000.0	
7	...	221.0	-159.0	567.0	380.0	601.0	0.0	
8	...	12211.0	11793.0	3719.0	3329.0	0.0	432.0	
9	...	0.0	13007.0	13912.0	0.0	0.0	0.0	

	PAY_AMT4	PAY_AMT5	PAY_AMT6	default.payment.next.month
0	0.0	0.0	0.0	1
1	1000.0	0.0	2000.0	1
2	1000.0	1000.0	5000.0	0
3	1100.0	1069.0	1000.0	0
4	9000.0	689.0	679.0	0
5	1000.0	1000.0	800.0	0
6	20239.0	13750.0	13770.0	0
7	581.0	1687.0	1542.0	0
8	1000.0	1000.0	1000.0	0
9	13007.0	1122.0	0.0	0

[10 rows x 25 columns]

Dataset Info :

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 30000 entries, 0 to 29999

Data columns (total 25 columns):

#	Column	Non-Null Count	Dtype
0	ID	30000 non-null	int64
1	LIMIT_BAL	30000 non-null	float64
2	SEX	30000 non-null	int64
3	EDUCATION	30000 non-null	int64
4	MARRIAGE	30000 non-null	int64
5	AGE	30000 non-null	int64
6	PAY_0	30000 non-null	int64
7	PAY_2	30000 non-null	int64
8	PAY_3	30000 non-null	int64
9	PAY_4	30000 non-null	int64
10	PAY_5	30000 non-null	int64
11	PAY_6	30000 non-null	int64
12	BILL_AMT1	30000 non-null	float64
13	BILL_AMT2	30000 non-null	float64

```

14 BILL_AMT3          30000 non-null float64
15 BILL_AMT4          30000 non-null float64
16 BILL_AMT5          30000 non-null float64
17 BILL_AMT6          30000 non-null float64
18 PAY_AMT1           30000 non-null float64
19 PAY_AMT2           30000 non-null float64
20 PAY_AMT3           30000 non-null float64
21 PAY_AMT4           30000 non-null float64
22 PAY_AMT5           30000 non-null float64
23 PAY_AMT6           30000 non-null float64
24 default.payment.next.month 30000 non-null int64
dtypes: float64(13), int64(12)
memory usage: 5.7 MB

```

```

In [ ]: # Preprocessing
X_fraud = df_fraud.drop('default.payment.next.month', axis=1)
y_fraud = df_fraud['default.payment.next.month']

# Scaling
scaler = StandardScaler()
X_fraud_scaled = scaler.fit_transform(X_fraud)

# Splitting
X_train_fraud, X_test_fraud, y_train_fraud, y_test_fraud = train_test_split(
    X_fraud_scaled, y_fraud, test_size=0.3, random_state=42
)
print(f"Training Data Shape: {X_train_fraud.shape}, Test Data Shape: {X_test_fraud.shape}")

```

Training Data Shape: (21000, 24), Test Data Shape: (9000, 24)

```

In [ ]: # Train and Evaluate Models
models_fraud = {
    '📊 Logistic Regression': LogisticRegression(max_iter=2000),
    '👤 K-Nearest Neighbors': KNeighborsClassifier(),
    '🌳 Decision Tree': DecisionTreeClassifier(),
}

metrics_fraud = {}

for model_name, model in models_fraud.items():
    model.fit(X_train_fraud, y_train_fraud)
    y_pred_fraud = model.predict(X_test_fraud)

    accuracy = accuracy_score(y_test_fraud, y_pred_fraud)
    precision = precision_score(y_test_fraud, y_pred_fraud, zero_division=0)
    recall = recall_score(y_test_fraud, y_pred_fraud, zero_division=0)
    f1 = f1_score(y_test_fraud, y_pred_fraud, zero_division=0)
    cm = confusion_matrix(y_test_fraud, y_pred_fraud)

    metrics_fraud[model_name] = {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'confusion_matrix': cm
    }

print("\nPerformance Metrics for Logistic Regression, KNN, and Decision Tree:")
for model_name, metric in metrics_fraud.items():
    print(f"\nModel : {model_name}\n")

```



```
print(f" Accuracy: {metric['accuracy']:.4f}")
print(f" Precision: {metric['precision']:.4f}")
print(f" Recall: {metric['recall']:.4f}")
print(f" F1 Score: {metric['f1_score']:.4f}")
print(f" \nConfusion Matrix:\n{metric['confusion_matrix']}\n")
print("-" * 80)
```

Performance Metrics for Logistic Regression, KNN, and Decision Tree:

Model : 📊 Logistic Regression

Accuracy: 0.8088
Precision: 0.6781
Recall: 0.2321
F1 Score: 0.3459

Confusion Matrix:

```
[[6824  216]
 [1505  455]]
```

Model : 👤 K-Nearest Neighbors

Accuracy: 0.7923
Precision: 0.5378
Recall: 0.3301
F1 Score: 0.4091

Confusion Matrix:

```
[[6484  556]
 [1313  647]]
```

Model : 🌳 Decision Tree

Accuracy: 0.7273
Precision: 0.3818
Recall: 0.4071
F1 Score: 0.3941

Confusion Matrix:

```
[[5748 1292]
 [1162  798]]
```

```
In [ ]: # ANN Model
ann_model = Sequential([
    Dense(64, input_dim=X_train_fraud.shape[1], activation='relu'),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

ann_model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
ann_model.fit(X_train_fraud, y_train_fraud, epochs=50, batch_size=32, verbose=1)

# ANN Predictions
```

```
y_pred_fraud_ann = (ann_model.predict(X_test_fraud) > 0.5).astype("int32")

accuracy_ann = accuracy_score(y_test_fraud, y_pred_fraud_ann)
precision_ann = precision_score(y_test_fraud, y_pred_fraud_ann, zero_division=0)
recall_ann = recall_score(y_test_fraud, y_pred_fraud_ann, zero_division=0)
f1_ann = f1_score(y_test_fraud, y_pred_fraud_ann, zero_division=0)
cm_ann = confusion_matrix(y_test_fraud, y_pred_fraud_ann)

metrics_fraud['🤖 Artificial Neural Network (Keras)'] = {
    'accuracy': accuracy_ann,
    'precision': precision_ann,
    'recall': recall_ann,
    'f1_score': f1_ann,
    'confusion_matrix': cm_ann
}
```

Epoch 1/50

/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
657/657 ————— 2s 2ms/step - accuracy: 0.7919 - loss: 0.5000
Epoch 2/50
657/657 ————— 1s 2ms/step - accuracy: 0.8190 - loss: 0.4382
Epoch 3/50
657/657 ————— 1s 2ms/step - accuracy: 0.8232 - loss: 0.4317
Epoch 4/50
657/657 ————— 2s 3ms/step - accuracy: 0.8247 - loss: 0.4248
Epoch 5/50
657/657 ————— 3s 3ms/step - accuracy: 0.8241 - loss: 0.4246
Epoch 6/50
657/657 ————— 2s 2ms/step - accuracy: 0.8253 - loss: 0.4217
Epoch 7/50
657/657 ————— 2s 2ms/step - accuracy: 0.8251 - loss: 0.4207
Epoch 8/50
657/657 ————— 1s 2ms/step - accuracy: 0.8273 - loss: 0.4159
Epoch 9/50
657/657 ————— 1s 2ms/step - accuracy: 0.8248 - loss: 0.4178
Epoch 10/50
657/657 ————— 1s 2ms/step - accuracy: 0.8244 - loss: 0.4120
Epoch 11/50
657/657 ————— 1s 2ms/step - accuracy: 0.8317 - loss: 0.4058
Epoch 12/50
657/657 ————— 1s 2ms/step - accuracy: 0.8228 - loss: 0.4177
Epoch 13/50
657/657 ————— 2s 3ms/step - accuracy: 0.8288 - loss: 0.4035
Epoch 14/50
657/657 ————— 2s 3ms/step - accuracy: 0.8292 - loss: 0.4076
Epoch 15/50
657/657 ————— 2s 2ms/step - accuracy: 0.8264 - loss: 0.4081
Epoch 16/50
657/657 ————— 1s 2ms/step - accuracy: 0.8265 - loss: 0.4077
Epoch 17/50
657/657 ————— 1s 2ms/step - accuracy: 0.8275 - loss: 0.4101
Epoch 18/50
657/657 ————— 2s 2ms/step - accuracy: 0.8298 - loss: 0.4058
Epoch 19/50
657/657 ————— 1s 2ms/step - accuracy: 0.8323 - loss: 0.3962
Epoch 20/50
657/657 ————— 1s 2ms/step - accuracy: 0.8319 - loss: 0.3993
Epoch 21/50
657/657 ————— 2s 2ms/step - accuracy: 0.8309 - loss: 0.4014
Epoch 22/50
657/657 ————— 3s 3ms/step - accuracy: 0.8303 - loss: 0.4033
Epoch 23/50
657/657 ————— 2s 2ms/step - accuracy: 0.8337 - loss: 0.3981
Epoch 24/50
657/657 ————— 1s 2ms/step - accuracy: 0.8288 - loss: 0.3989
Epoch 25/50
657/657 ————— 1s 2ms/step - accuracy: 0.8311 - loss: 0.4003
Epoch 26/50
657/657 ————— 1s 2ms/step - accuracy: 0.8349 - loss: 0.3937
Epoch 27/50
657/657 ————— 2s 2ms/step - accuracy: 0.8306 - loss: 0.3964
Epoch 28/50
657/657 ————— 1s 2ms/step - accuracy: 0.8331 - loss: 0.3930
Epoch 29/50
657/657 ————— 1s 2ms/step - accuracy: 0.8350 - loss: 0.3942
Epoch 30/50
657/657 ————— 2s 3ms/step - accuracy: 0.8332 - loss: 0.3947
Epoch 31/50
```

```

657/657 ————— 3s 3ms/step - accuracy: 0.8373 - loss: 0.3882
Epoch 32/50
657/657 ————— 1s 2ms/step - accuracy: 0.8372 - loss: 0.3870
Epoch 33/50
657/657 ————— 2s 2ms/step - accuracy: 0.8352 - loss: 0.3925
Epoch 34/50
657/657 ————— 3s 2ms/step - accuracy: 0.8349 - loss: 0.3863
Epoch 35/50
657/657 ————— 1s 2ms/step - accuracy: 0.8373 - loss: 0.3841
Epoch 36/50
657/657 ————— 2s 2ms/step - accuracy: 0.8408 - loss: 0.3803
Epoch 37/50
657/657 ————— 2s 2ms/step - accuracy: 0.8390 - loss: 0.3851
Epoch 38/50
657/657 ————— 3s 3ms/step - accuracy: 0.8390 - loss: 0.3774
Epoch 39/50
657/657 ————— 1s 2ms/step - accuracy: 0.8400 - loss: 0.3830
Epoch 40/50
657/657 ————— 2s 2ms/step - accuracy: 0.8438 - loss: 0.3792
Epoch 41/50
657/657 ————— 1s 2ms/step - accuracy: 0.8386 - loss: 0.3770
Epoch 42/50
657/657 ————— 1s 2ms/step - accuracy: 0.8404 - loss: 0.3811
Epoch 43/50
657/657 ————— 1s 2ms/step - accuracy: 0.8427 - loss: 0.3724
Epoch 44/50
657/657 ————— 1s 2ms/step - accuracy: 0.8386 - loss: 0.3768
Epoch 45/50
657/657 ————— 3s 2ms/step - accuracy: 0.8424 - loss: 0.3757
Epoch 46/50
657/657 ————— 3s 3ms/step - accuracy: 0.8433 - loss: 0.3741
Epoch 47/50
657/657 ————— 2s 2ms/step - accuracy: 0.8415 - loss: 0.3784
Epoch 48/50
657/657 ————— 1s 2ms/step - accuracy: 0.8434 - loss: 0.3726
Epoch 49/50
657/657 ————— 3s 2ms/step - accuracy: 0.8413 - loss: 0.3797
Epoch 50/50
657/657 ————— 1s 2ms/step - accuracy: 0.8425 - loss: 0.3690
282/282 ————— 1s 2ms/step

```

```

In [ ]: from tabulate import tabulate

# Summary of all models
summary_data = []

for model_name, metric in metrics_fraud.items():
    summary_data.append([
        model_name,
        metric['accuracy'],
        metric['precision'],
        metric['recall'],
        metric['f1_score']
    ])

headers = ["Model", "Accuracy", "Precision", "Recall", "F1 Score"]

print("\nFinal Performance Metrics (Tabular Form):")
print(tabulate(summary_data, headers=headers, tablefmt="fancy_grid", floatfmt=".

```

Final Performance Metrics (Tabular Form):

Model F1 Score	Accuracy	Precision	Recall
 Logistic Regression 0.3459	0.8088	0.6781	0.2321
 K-Nearest Neighbors 0.4091	0.7923	0.5378	0.3301
 Decision Tree 0.3941	0.7273	0.3818	0.4071
 Artificial Neural Network (Keras) 0.4424	0.8076	0.5995	0.3505

Metro Interstate Traffic Volume Data

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tabulate import tabulate

In [ ]: data = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/ML_7/p3_metro.csv')
print("\nData Preview : \n\n", data.head(10))
print()
print("-" * 100)
print("\nDataset Info : \n")
data.info()
print()

data.drop(['holiday', 'weather_main', 'weather_description', 'date_time'], axis=

data.fillna(data.mean(), inplace=True)

print("-" * 100)

print("\nProcessed Data : \n\n", data.head(10))

X = data.drop('traffic_volume', axis=1)
y = data['traffic_volume']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_
```

Data Preview :

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main \
0	NaN	288.28	0.0	0.0	40	Clouds
1	NaN	289.36	0.0	0.0	75	Clouds
2	NaN	289.58	0.0	0.0	90	Clouds
3	NaN	290.13	0.0	0.0	90	Clouds
4	NaN	291.14	0.0	0.0	75	Clouds
5	NaN	291.72	0.0	0.0	1	Clear
6	NaN	293.17	0.0	0.0	1	Clear
7	NaN	293.86	0.0	0.0	1	Clear
8	NaN	294.14	0.0	0.0	20	Clouds
9	NaN	293.10	0.0	0.0	20	Clouds

	weather_description	date_time	traffic_volume
0	scattered clouds	2012-10-02 09:00:00	5545
1	broken clouds	2012-10-02 10:00:00	4516
2	overcast clouds	2012-10-02 11:00:00	4767
3	overcast clouds	2012-10-02 12:00:00	5026
4	broken clouds	2012-10-02 13:00:00	4918
5	sky is clear	2012-10-02 14:00:00	5181
6	sky is clear	2012-10-02 15:00:00	5584
7	sky is clear	2012-10-02 16:00:00	6015
8	few clouds	2012-10-02 17:00:00	5791
9	few clouds	2012-10-02 18:00:00	4770

Dataset Info :

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 48204 entries, 0 to 48203

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	holiday	61 non-null	object
1	temp	48204 non-null	float64
2	rain_1h	48204 non-null	float64
3	snow_1h	48204 non-null	float64
4	clouds_all	48204 non-null	int64
5	weather_main	48204 non-null	object
6	weather_description	48204 non-null	object
7	date_time	48204 non-null	object
8	traffic_volume	48204 non-null	int64

dtypes: float64(3), int64(2), object(4)

memory usage: 3.3+ MB

Processed Data :

	temp	rain_1h	snow_1h	clouds_all	traffic_volume
0	288.28	0.0	0.0	40	5545
1	289.36	0.0	0.0	75	4516
2	289.58	0.0	0.0	90	4767
3	290.13	0.0	0.0	90	5026
4	291.14	0.0	0.0	75	4918
5	291.72	0.0	0.0	1	5181

6	293.17	0.0	0.0	1	5584
7	293.86	0.0	0.0	1	6015
8	294.14	0.0	0.0	20	5791
9	293.10	0.0	0.0	20	4770

```
In [ ]: # Multi-Linear regression

linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)
```

```
In [ ]: # Transform features to polynomial degree 2
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Polynomial Regression
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

y_pred_poly = poly_model.predict(X_test_poly)

mse_poly = mean_squared_error(y_test, y_pred_poly)
r2_poly = r2_score(y_test, y_pred_poly)
```

```
In [ ]: # ANN

ann_model = Sequential()
ann_model.add(Dense(64, input_dim=X_train.shape[1], activation='relu'))
ann_model.add(Dense(32, activation='relu'))
ann_model.add(Dense(1, activation='linear'))

ann_model.compile(optimizer='adam', loss='mse', metrics=['mse'])

ann_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)

y_pred_ann = ann_model.predict(X_test).flatten()

mse_ann = mean_squared_error(y_test, y_pred_ann)
r2_ann = r2_score(y_test, y_pred_ann)
```

Epoch 1/50

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1055/1055 ————— 5s 3ms/step - loss: 7313807.0000 - mse: 7313807.0000
Epoch 2/50
1055/1055 ————— 4s 2ms/step - loss: 3870035.0000 - mse: 3870035.0000
Epoch 3/50
1055/1055 ————— 2s 2ms/step - loss: 3885662.0000 - mse: 3885662.0000
Epoch 4/50
1055/1055 ————— 2s 2ms/step - loss: 3866673.2500 - mse: 3866673.2500
Epoch 5/50
1055/1055 ————— 3s 2ms/step - loss: 3866383.0000 - mse: 3866383.0000
Epoch 6/50
1055/1055 ————— 3s 3ms/step - loss: 3855736.0000 - mse: 3855736.0000
Epoch 7/50
1055/1055 ————— 4s 2ms/step - loss: 3850028.2500 - mse: 3850028.2500
Epoch 8/50
1055/1055 ————— 2s 2ms/step - loss: 3855265.0000 - mse: 3855265.0000
Epoch 9/50
1055/1055 ————— 2s 2ms/step - loss: 3834307.7500 - mse: 3834307.7500
Epoch 10/50
1055/1055 ————— 3s 2ms/step - loss: 3850098.7500 - mse: 3850098.7500
Epoch 11/50
1055/1055 ————— 4s 3ms/step - loss: 3823184.5000 - mse: 3823184.5000
Epoch 12/50
1055/1055 ————— 3s 2ms/step - loss: 3835480.2500 - mse: 3835480.2500
Epoch 13/50
1055/1055 ————— 3s 2ms/step - loss: 3845892.2500 - mse: 3845892.2500
Epoch 14/50
1055/1055 ————— 2s 2ms/step - loss: 3830237.7500 - mse: 3830237.7500
Epoch 15/50
1055/1055 ————— 3s 2ms/step - loss: 3837156.2500 - mse: 3837156.2500
Epoch 16/50
1055/1055 ————— 4s 3ms/step - loss: 3814075.2500 - mse: 3814075.2500
Epoch 17/50
1055/1055 ————— 3s 2ms/step - loss: 3814053.5000 - mse: 3814053.5000
Epoch 18/50
1055/1055 ————— 2s 2ms/step - loss: 3822511.5000 - mse: 3822511.5000
Epoch 19/50
1055/1055 ————— 3s 2ms/step - loss: 3859986.5000 - mse: 3859986.5000
Epoch 20/50
1055/1055 ————— 2s 2ms/step - loss: 3788010.0000 - mse: 3788010.0000
Epoch 21/50
```



```
1055/1055 ————— 4s 3ms/step - loss: 3840582.5000 - mse: 3840582.5000
Epoch 22/50
1055/1055 ————— 3s 2ms/step - loss: 3811478.7500 - mse: 3811478.7500
Epoch 23/50
1055/1055 ————— 3s 2ms/step - loss: 3845764.0000 - mse: 3845764.0000
Epoch 24/50
1055/1055 ————— 2s 2ms/step - loss: 3807467.7500 - mse: 3807467.7500
Epoch 25/50
1055/1055 ————— 2s 2ms/step - loss: 3811012.0000 - mse: 3811012.0000
Epoch 26/50
1055/1055 ————— 3s 2ms/step - loss: 3823007.2500 - mse: 3823007.2500
Epoch 27/50
1055/1055 ————— 4s 2ms/step - loss: 3817838.2500 - mse: 3817838.2500
Epoch 28/50
1055/1055 ————— 3s 2ms/step - loss: 3811951.2500 - mse: 3811951.2500
Epoch 29/50
1055/1055 ————— 3s 2ms/step - loss: 3811791.2500 - mse: 3811791.2500
Epoch 30/50
1055/1055 ————— 3s 2ms/step - loss: 3823318.5000 - mse: 3823318.5000
Epoch 31/50
1055/1055 ————— 4s 3ms/step - loss: 3817771.2500 - mse: 3817771.2500
Epoch 32/50
1055/1055 ————— 2s 2ms/step - loss: 3825236.5000 - mse: 3825236.5000
Epoch 33/50
1055/1055 ————— 2s 2ms/step - loss: 3821275.5000 - mse: 3821275.5000
Epoch 34/50
1055/1055 ————— 2s 2ms/step - loss: 3799360.0000 - mse: 3799360.0000
Epoch 35/50
1055/1055 ————— 2s 2ms/step - loss: 3811672.2500 - mse: 3811672.2500
Epoch 36/50
1055/1055 ————— 3s 2ms/step - loss: 3830501.0000 - mse: 3830501.0000
Epoch 37/50
1055/1055 ————— 5s 2ms/step - loss: 3807758.5000 - mse: 3807758.5000
Epoch 38/50
1055/1055 ————— 2s 2ms/step - loss: 3809825.5000 - mse: 3809825.5000
Epoch 39/50
1055/1055 ————— 3s 2ms/step - loss: 3824985.7500 - mse: 3824985.7500
Epoch 40/50
1055/1055 ————— 2s 2ms/step - loss: 3801034.2500 - mse: 3801034.2500
Epoch 41/50
```

```

1055/1055 ————— 3s 3ms/step - loss: 3831820.2500 - mse: 3831820.2500
Epoch 42/50
1055/1055 ————— 4s 2ms/step - loss: 3820990.7500 - mse: 3820990.7500
Epoch 43/50
1055/1055 ————— 2s 2ms/step - loss: 3807509.5000 - mse: 3807509.5000
Epoch 44/50
1055/1055 ————— 3s 2ms/step - loss: 3826045.7500 - mse: 3826045.7500
Epoch 45/50
1055/1055 ————— 2s 2ms/step - loss: 3778975.0000 - mse: 3778975.0000
Epoch 46/50
1055/1055 ————— 3s 3ms/step - loss: 3813321.5000 - mse: 3813321.5000
Epoch 47/50
1055/1055 ————— 4s 2ms/step - loss: 3809996.5000 - mse: 3809996.5000
Epoch 48/50
1055/1055 ————— 3s 2ms/step - loss: 3847804.5000 - mse: 3847804.5000
Epoch 49/50
1055/1055 ————— 3s 2ms/step - loss: 3801275.7500 - mse: 3801275.7500
Epoch 50/50
1055/1055 ————— 3s 3ms/step - loss: 3789996.7500 - mse: 3789996.7500
452/452 ————— 1s 3ms/step

```

```

In [ ]: # Results

results = [
    ['🔥 Multi-Linear Regression', mse_linear, r2_linear],
    ['★ Polynomial Regression (Degree 2)', mse_poly, r2_poly],
    ['🤖 Artificial Neural Network', mse_ann, r2_ann]
]

print("\nComparison of Models : \n")
print(tabulate(results, headers=['Model', 'MSE', 'R² Score'], tablefmt='fancy_gr

```

Comparison of Models :

Model	MSE	R² Score
🔥 Multi-Linear Regression	3.87516e+06	0.025468
★ Polynomial Regression (Degree 2)	3.83033e+06	0.0367421
🤖 Artificial Neural Network	3.83208e+06	0.0363016